PES UNIVERSITY

Software Engineering Laboratory

LAB 5

STATIC CODE ANALYSIS

Campus Parking Permit Manager

Student name	I. V. Thanvitha
SRN	PES2UG24CS197
Section	D
Experiment	Lab 5 - Static Code Analysis
Assigned file	parking_system.py
Date	14 October 2026

Submission summary

Analyzed and corrected the assigned Python parking permit application using Pylint, Bandit and Flake8. Documented 13 issues, repaired the program, and verified the updated version with 13 automated unit tests.

Pylint	Bandit	Flake8	Unit tests
10.00 / 10	0 issues	0 violations	13 passed

Source repository: github.com/thanvitha-ippagunta/L5_10_parking_permits

1. Objective and experiment overview

To improve Python code quality, security and style through static analysis with **Pylint, Bandit and Flake8**; identify, document and fix at least four issues, prioritizing high- and medium-priority findings.

The assigned program is a campus parking permit manager. It registers vehicles in zones A, B and C, creates permit numbers, checks expiry dates, scans zones for invalid vehicles, revokes permits and saves records in JSON.

2. Tools and their purpose

Tool	Purpose in this experiment
Pylint	Checks code quality and common Python defects: mutable defaults, naming, missing docstrings and file handling.
Bandit	Scans Python source for security-sensitive coding patterns such as predictable random-number generation.
Flake8	Checks style and simple program issues including PEP 8 formatting and bare exception handling.
unittest	Runs behavioral regression checks after the static-analysis findings are fixed.

3. Procedure performed

1. Cloned the assigned repository and inspected the original `parking_system.py`.

2. Examined quality, security and functional defects; documented 13 findings with their original line references and correction approaches.

3. Corrected the code and introduced targeted unit tests in `tests/test_parking_system.py`.

4. Installed the static-analysis tools in a Python virtual environment and reran Pylint, Bandit, Flake8 and the unit test suite.

5. Saved final reports as `pylint_report.txt`, `bandit_report.txt` and `flake8_report.txt`.

Commands used (Windows PowerShell)

```
..\.venv\Scripts\python.exe -m pylint parking_system.py > pylint_report.txt
..\.venv\Scripts\python.exe -m bandit -r parking_system.py > bandit_report.txt
..\.venv\Scripts\python.exe -m flake8 parking_system.py > flake8_report.txt
```

Evidence scope: The final tool outputs and unit-test run are recorded in terminal screenshots. Original pre-fix score reports were not provided, so no numerical before-versus-after score is claimed. The issue table uses original starter line numbers from the documented review.

4. Filled-out table of identified issues

Issue documentation - Part 1 of 2. The **Line(s)** column refers to the **original starter program**, not the updated file. Priority is assessment of repair importance, not necessarily the exact tool-reported severity.

Issue	Type	Line(s)	Description	Fix Approach
Predictable permit IDs	Security (Med.)	2, 9	random.randint generated predictable-looking IDs.	Replaced with secrets.randbelow(); added a collision check.
Broad except clauses	Bug (High)	14-15; 19-20; 26-27; 39-40	Bare except clauses swallowed unrelated failures and hid malformed records.	Caught relevant ValueError, KeyError, TypeError, file/JSON and Unicode exceptions.
Invalid expiry accepted	Security (High)	23-27, 55	A date written as 01/01/2020 did not match the expected format; the error path treated it as valid.	Validate ISO dates and treat missing or malformed expiries as expired.
No plate/zone validation	Correctness (High)	7-12, 52-54	Accepted empty or missing plates and unsupported zone Z.	Normalize valid plates/zones and reject bad registrations.
Mutable default list	Bug (Med.)	7, 12	zone_records=[] could retain entries across calls.	Use a None default; append only when a list is supplied.
Expiry checked at midnight	Logic (Med.)	24-25	datetime.now() made a permit expiring today appear expired during the day.	Compare date values; permit remains valid on its expiry date.
Unmanaged file handles	Resource (Med.)	38, 43-45	open() calls did not use context managers.	Use with open(..., encoding="utf-8") for reads and writes.

The high-priority fixes addressed invalid permit expiry, overly broad exception handling and input validation. These directly affect whether the program accepts a vehicle as authorized.

4. Filled-out issue table (continued)

Issue documentation - Part 2 of 2. All 13 listed findings were addressed in the submitted Python file.

Issue	Type	Line(s)	Description	Fix Approach
File read errors erase data	Data integrity (Med.)	35-40	Every read error reset permits to {}, risking overwrite of valid data.	Missing file creates an empty registry; malformed JSON raises an error.
Unvalidated JSON data	Correctness (Med.)	38	Malformed saved records could be trusted when loaded.	Validate saved zones, permit IDs and dates; skip invalid entries with warnings.
Duplicate registration reset	Logic (Med.)	11	Registering the same plate again silently replaced its permit/expiry.	Keep the existing permit and original expiry.
Invalid scan zone	Correctness (Low)	29-34	scan_zone allowed unsupported zones.	Reject invalid scan zones with ValueError.
Naming/docs/formatting	Style (Low)	47 and others	Non-PEP8 setExpiry, missing docstrings and long statements.	Use set_expiry, docstrings, clearer formatting and f-strings.
Runs when imported	Testability (Med.)	63	Original main() executed during import.	Use if __name__ == "__main__": main().

Selected corrected source patterns

Mutable list default and secure permit identifier:

```
def register_permit(vehicle_plate, zone_id, zone_records=None):
    ...
permit_no = f"P{secrets.randbelow(90_000) + 10_000}"
```

Fail-closed expiry evaluation and guarded entry point:

```
expiry = date.fromisoformat(expiry_text)
return expiry < date.today()

if __name__ == "__main__":
    main()
```

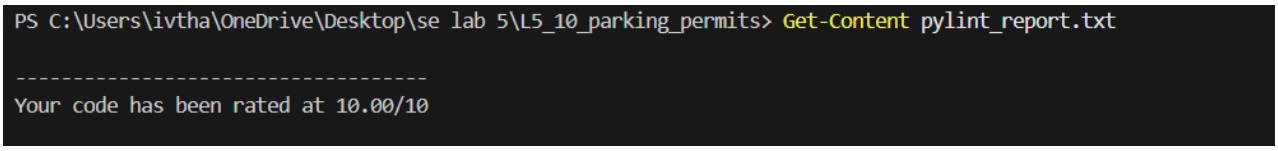
All table entries are documented in the repository `issues.md`. The corrected source also validates JSON records and uses context-managed file operations.

5. Post-fix static analysis - actual results

The following results were obtained in the local terminal after applying the fixes to `parking_system.py`.

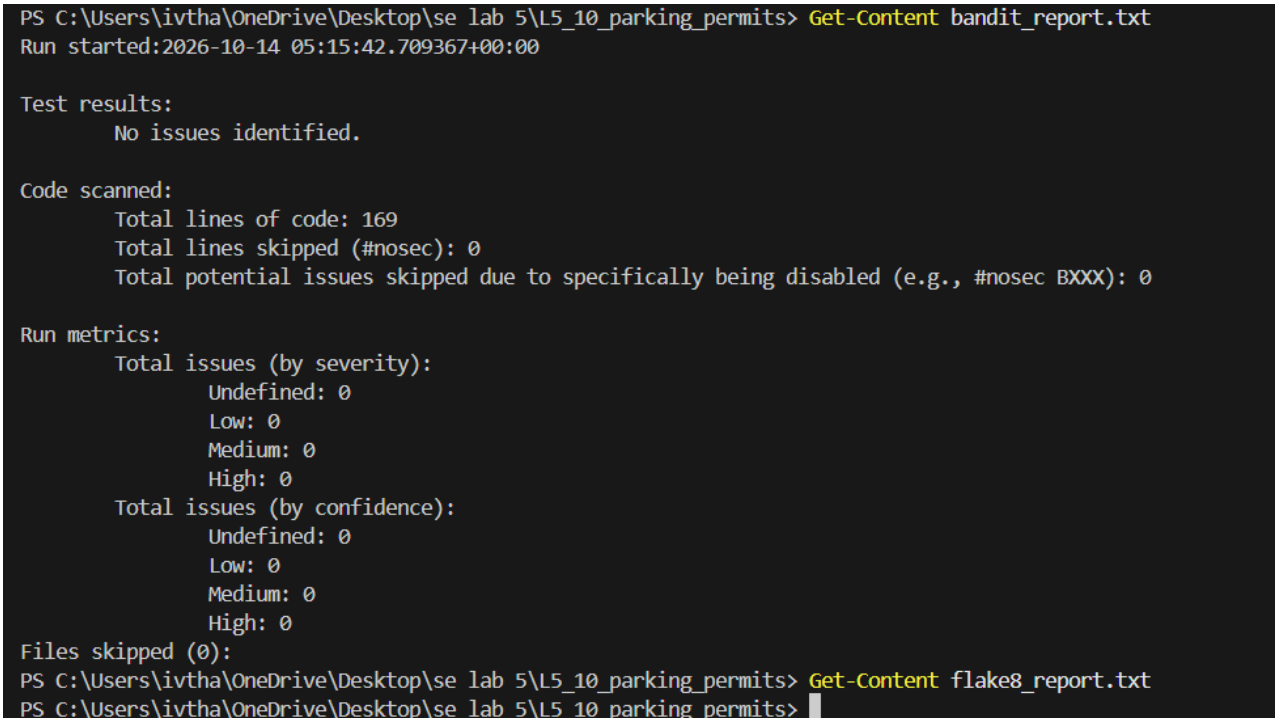
Analysis tool	Observed result	Interpretation
Pylint	10.00 / 10	Code quality scan completed at the maximum rating.
Bandit	No issues identified	0 high-, medium- or low-severity findings in the final scan.
Flake8	Empty report	No reported style violations after execution.

Figure 1. Pylint terminal output



Terminal output showing: "Your code has been rated at 10.00/10."

Figure 2. Bandit scan and Flake8 output



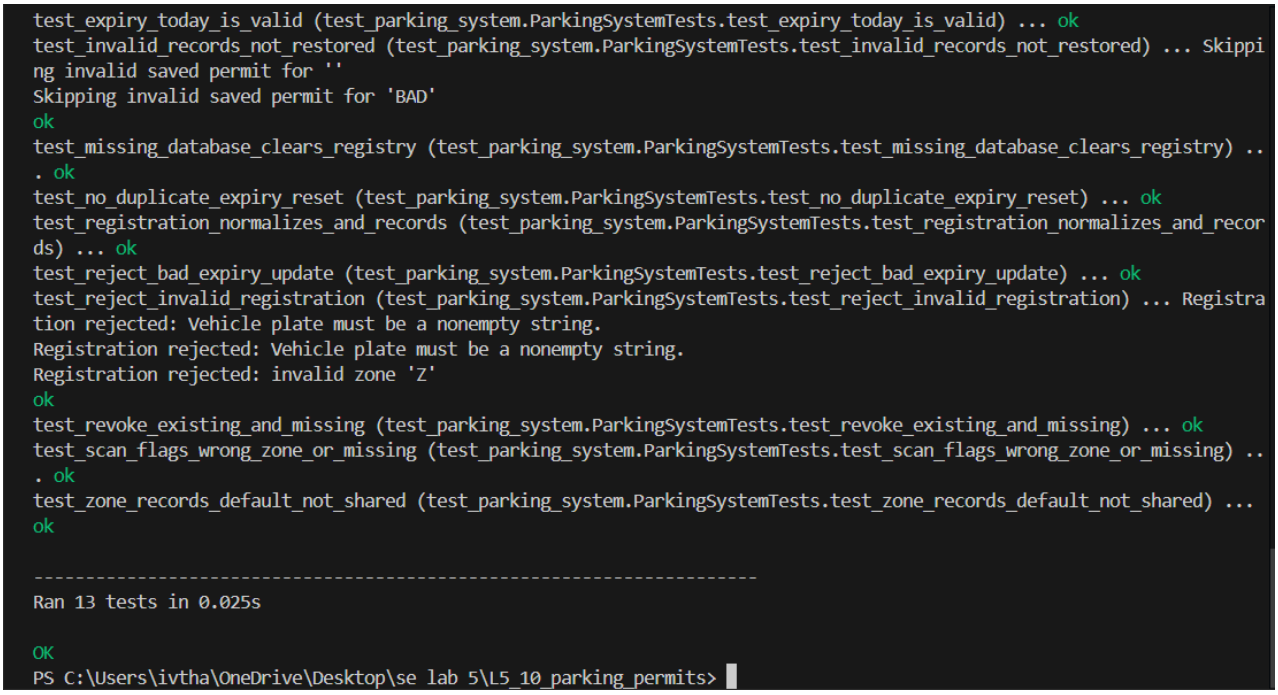
Bandit reports zero findings. Flake8 returns no output when its report is displayed. Screenshots were provided from the student's local run on 14 October 2026.

6. Automated unit-test verification

Unit tests check the runtime behavior that static tools cannot guarantee. The corrected program was tested for permit registration, revocation, zones, expiry boundaries and JSON loading.

Verification	Observed result
Command	python -m unittest discover -s tests -v
Number of tests	13
Final result	OK - all 13 passed
Separate compilation check	python -m py_compile parking_system.py - passed

Figure 3. Local unit-test execution



The terminal output ends with "Ran 13 tests" and "OK". This screenshot records the student's local verification.

Behaviors covered

The tests cover invalid registrations and zones, plate normalization, avoiding duplicate expiry resets, valid expiry on the current date, malformed expiry handling, revocation, zone scans, isolation of mutable-list defaults, saving/loading JSON, skipping malformed saved records and missing/corrupt database handling.

7. Reflection questions and answers

The uploaded four-page handout mentions reflection questions but does not include their exact wording. The four questions below are based on its stated learning outcomes and match the accompanying reflection.md.

Q1. What is static code analysis, and why is it useful?

Static code analysis examines source code without running the application. It helps identify unsafe patterns, possible bugs and style violations early. In this project, reviewing bare exception handlers, mutable defaults and predictable ID generation helped improve maintainability and reduce incorrect permit decisions.

Q2. How do Pylint, Flake8 and Bandit differ?

Pylint reviews code quality, suspicious constructs and naming practices. Flake8 checks PEP 8 conventions and basic errors such as a bare except clause. Bandit focuses on security-sensitive Python patterns, such as using ordinary pseudo-random numbers where unpredictable IDs are preferred. Using all three gives broader coverage than any single tool.

Q3. Which issues were the most important to fix, and what changed?

The most important issue was incorrect expiry validation: an invalid date could be treated as not expired because a broad exception returned a permissive result. I changed expiry handling to validate ISO dates and fail closed. I also validated plate and zone input, replaced the shared mutable default, used secrets for permit IDs and made JSON file handling safer.

Q4. What did you learn from verifying the fixes?

Clean static-analysis results do not prove complete functional correctness. I therefore reran Pylint, Bandit and Flake8 and also executed the regression suite. The results were Pylint 10.00/10, Bandit zero findings, Flake8 no reported violations and 13 tests passed. I learned to combine automated source checks with behavior-based testing.

8. Conclusion

The parking permit program was cleaned, validated and retested. Thirteen documented issues were addressed, exceeding the handout's minimum requirement of four fixes. Final local results show no findings from the three static-analysis tools and all 13 unit tests passing. Static-analysis results apply to this tested version and do not guarantee the absence of every possible defect.

References and submission evidence

1. PES University, *Lab 5: Static Code Analysis*, student handout (provided).

2. Student repository: https://github.com/thanvitha-ippagunta/L5_10_parking_permits

3. Repository files: `issues.md`, `reflection.md`, `pylint_report.txt`, `bandit_report.txt`, `flake8_report.txt`, and `tests/`.